

Part 1: Prescraping Process

Context:

SearchPageClient.tsx is the file that defines the SearchPage meaning its the highest page in the pecking order.

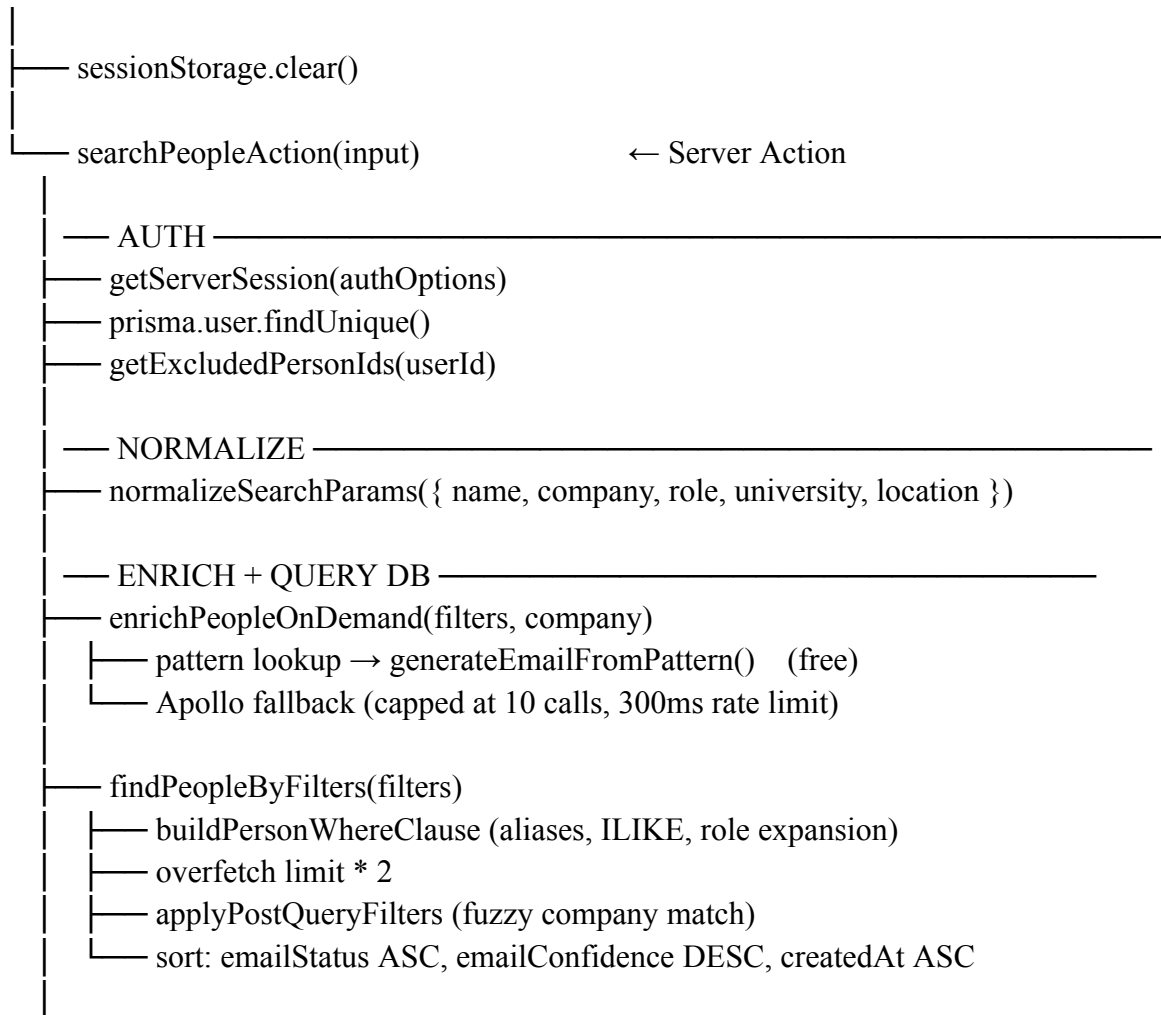
Server Actions are just functions that are run on the server.

UI: User Clicks Search

- 1) In SearchForm.tsx, in handleSubmit(), we collect search results and call the parent onSearch() function (a function passed into SearchForm.tsx from SearchPageClient.tsx). onSearch() is actually defined by handleSearch() in SearchPageClient.tsx. While handleSearch() is the actual function, it is passed in as the parameter to the SearchForm component.
- 2) handleSearch() clears the state and then begins the server process of discovery

Initial DB Check For Results

Client: handleSearch(params)

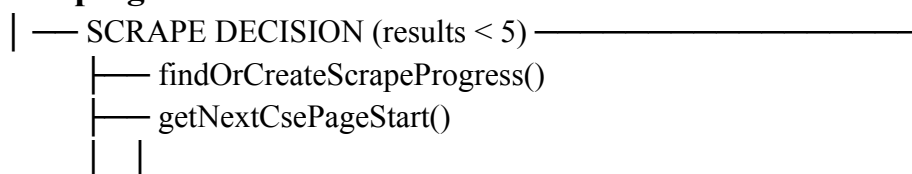


The **Auth Section** does 3 things. First, it gets the user session and returns it to the user via getServerSession(). It basically tells the server which user is accessing it. Next, it calls prisma.user.findUnique(), which gets all the relevant user data associated with handleSearch(). This includes name, email, templates, etc. Finally, the getExcludedPersonKeys() finds people whom the user has already sent emails to/hidden away, eliminating them from the potential pool.

- 1) The **Normalize Section** simply just formats the user input and makes sure its ready to be used in other sections of the code. It simply trims whitespace, makes everything lowercase, and then returns null for sections that the user doesn't specify.
- 2) The **Query DB** step is where we find out how many valid results we have before scraping. It calls the **findPeopleByFilters(filters)** function.

- getCompanySearchTerms()/getRoleSearchTerms()- These two functions basically return a list of company aliases and role aliases. That way, when building the where clause for the db, it is able to capture all the different names/roles a company/ role can be referred to by.
- ExcludePersonIds Filter- This is a new filter I implemented in the buildPersonWhereClause() function. I basically just moved this step from post query filtering to the actual query instead.
- enrichPeopleOnDemand()- This function queries the database for people who match the user's search criteria and haven't had their email checked. It then either pattern matches their email or makes an Apollo call to find their email and populates the db with their new email.
- buildPersonWhereClause()- This defines the where clause for the database query (basically builds the db query). The function: 1) excludes people that are currently being shown on the page, 2) excludes people that match any of the company/role alias names, 3) excludes people that don't go to the university, 4) filters people that don't have emails via the emailDeliverable column. This is apart of a bigger function called **findPeopleByFilters(filters)**.
- prisma.person.findMany()- This is where the **offset bullshit** is coming from, and I just removed it. This function simply runs the query we built, performs email rankings, and returns the results. Now, because we are tracking who is being shown on screen, we don't need any offset measures. We simply get all the people that match the initial criteria and filter out the people taht are already being shown on screen.
- companiesMatch()- This is a post query filter. After the query is run, the way getCompanySearchTerms() is that it filters out a lot of the companies that don't match the criteria, but strictly to maintain db speed so the db isn't sorting a fuck ton of rows. Now, the companiesMatch() is what is actually used for strict filtering. If the user's company doesn't match any of the potential alias, then they are filtered out. This filtering can't be done in the db because of the way Prisma works. Note: we overfetch front he query to account for potential filter out candidates.

Scrapping Process



- 0 results + not scraped: SYNC scrape 
 - processRefreshBatch(input, nextPage)
 - getCompanyAliases()
 - discoverLinkedInProfiles() ← CSE API
 - findPeopleByLinkedInUrls()
 - scrapeLinkedInProfiles() ← LinkedIn
 - saveScrapedProfile() for each
 - updateScrapeProgress()
 - enrichPeopleOnDemand(filters, company) ← enrich new
 - findPeopleByFilters(...) ← re-query
- 1+ results or already scraped: return immediately
 - (frontend triggers prescrape after response)

— RANK + BUILD —

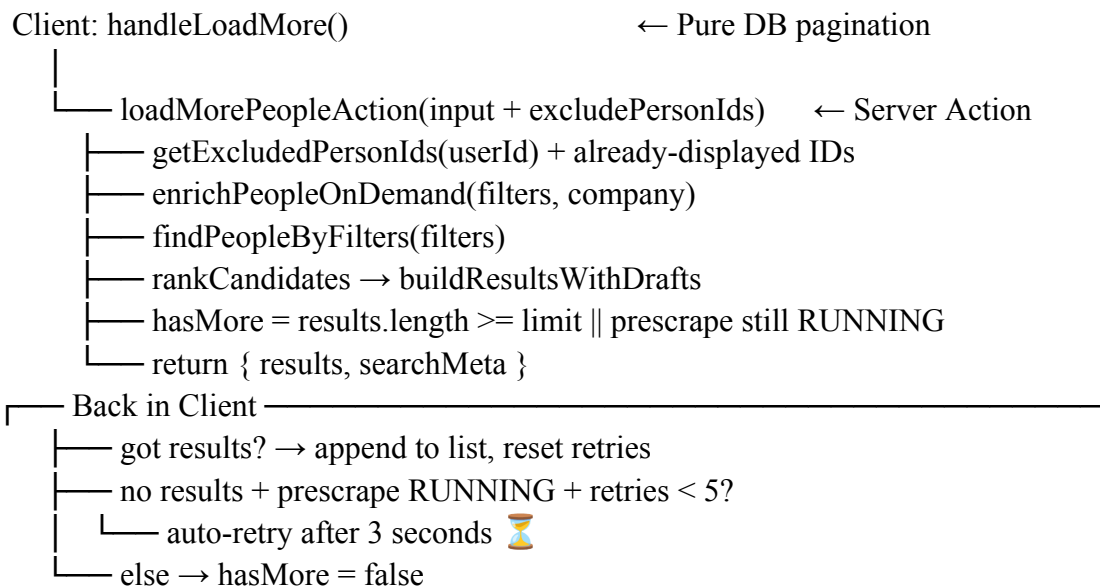
- rankCandidates(searchCriteria, people, mapper, limit)
- buildResultsWithDrafts(rankedPeople, userId, templateId, user)
 - prisma.userCandidate.upsert()
 - generateEmailDraft() ← template replacement
 - prisma.emailDraft.upsert()

— HASMORE + LOG —

- hasMore = people.length >= limit || cseHasMorePages
- prisma.searchLog.create()
- return { results, searchMeta: { hasMore, ... }, remainingDaily }

— Back in Client —

- display results + save to sessionStorage
- HTTP POST /api/prescrape (fire-and-forget )
 - prescrapeAction(input) ← Server Action
 - set prescrapeStatus = RUNNING
 - loop (up to 4 pages):
 - getNextCsePageStart()
 - processRefreshBatch() ← CSE + LinkedIn
 - updateScrapeProgress()
 - break if < 5 valid profiles (exhausted)
 - set prescrapeStatus = DONE



Scrape Decision- These two functions examine the search query. If returns which page we were on (assuming we didn't reach the 5 page limit). If we did, then it creates a new search and scrapes 5 pages in the background.

Scraping Process Summary:

If the search has been scraped before, we simply just return results from the database. If we haven't made the search before, we simply scrape the 5 pages associated with that search and add them to the db sequentially. That way, when the user clicks load more, there should be results that have been added to the db for the user to see.

Scraper Process in Depth

- 1) processRefreshBatch(input, nextPage)
 - `discoverLinkedInProfiles()`- This is the function that calls the CSE and returns all the linkedin profiles on the page. Note: I added functionality where it checks if the search matches the user's company search. If not, we skip that result (limit CSE calls).
 - `getCompanyAliases()`- runs the company through Groq and asks it to expand the company names in the search (So now Goldman Sachs and GS are both in the search). Then it caches company aliases so we don't have to call Groq for repeat companies.
 - `findPeopleByLinkedInUrls()`- This function checks the db to see if these people already exist. If they do skip them.
 - `scrapeLinkedInProfiles()`- this function takes in an entire page worth of CSE data and scrapes the profiles in 1 batch.
 - `savedScrapedProfiles()`- upserts (to avoid race conditions) new people into the db.
- 2) updateScraperProgress()- This function updates the pages we have scraped. If we have reached page 5, stop scraping.
- 3) enrichPeopleOnDemand()- Same enrich step from earlier. Get emails for people via pattern matching or save people to db without emails if they don't match company.
- 4) findPeopleByFilters()- same db query function from earlier.

Load More

Don't need to explain the function associated. Way it works is that if its a background search, the each page that is processed from CSE is added to the db immediately. That way, when the user clicks load more, there is stuff that appears.

If it is a fresh search, the scraper will return the first page of results as soon as it finishes, and then in the background the scraper is populating the db with more results.

Changes Since The Last Refactor

Changes Made in Query DB Step:

1. b8d035f — Persist emailDeliverable from Apollo in bootstrapCompanyPattern. The field was available on the result but never saved to the Person row.

Backfilled 239 existing rows to true.

2. 0891512 — Three related improvements to search:

- Removed offset pagination — pagination now works purely through excludePersonIds (NOT IN), eliminating skipped people during background scraping
- Moved sent/hidden exclusion to DB level — replaced in-memory fullName_company key filtering with a DB-level id NOT IN, reducing overfetch waste
- Fixed short company alias matching — changed equals to startsWith for aliases like "ey" so entities like "EY Advisory" are no longer missed

Changes Made in the Path Decision Step

- Got rid of Path B in the search, so now we simply run the scraper parallel to the user query (if we have 1-4 results) or we just run the scraper entirely in the background if we

Scraper Changes:

- 1) To limit Apollo API calls, for searches where CSE returns people that aren't a part of the target company, instead of calling Apollo on them, we simply add them to the db without an email. That way, when that person's actual company gets searched up a user, then the email will be populated assuming that person meets all the filters.
- 2) When the db query is made on a company, if they don't have an email, and apollo call hasn't been made on it, and they match the company that is being queried, then it will extract that person and get their email from Apollo, updating their Apollo status, and add their email to the db and populate that person.
- 3) Apollo pattern learning has been removed. We have to manually add patterns to the db from now on.
- 4) Now, instead of the background scrape being triggered by load more, now, on a new search, we scrape 5 pages in the background. The load more button simply just calls the

database from now on. Now, scrapes only occur on the initial search. If the search has occurred, then we simply just query database for results.

Vector Embedding Role Ranking System

- 1) Each time a person is added to the db, updatePersonRoleEmbedding() is called on them. This vectorizes their role and adds it to the database in that column.
- 2)